Zeno Fuchs

Mein Ferien Projekt: Cross Launcher für Games

Projektidee(was?):

- **Konzept:** Eine Desktop-App, die verschiedene Gaming-Plattformen (z. B. Steam, Epic Games, Ubisoft Connect, Rockstar Games) an einem zentralen Ort bündelt.

- **Kernfunktion:** Automatische Erkennung und Anzeige aller lokal installierten Spiele auf dem PC. Der Nutzer kann seine Spiele direkt aus der App heraus starten, ohne die einzelnen Launcher manuell durchsuchen zu müssen.

- **Erweiterungs-Funktion:** Einbindung der jeweiligen Bibliotheken (Libraries), um auch nicht installierte Spiele anzuzeigen und den Download/die Weiterleitung über den jeweiligen Launcher auszulösen.

Was erreichen (wie viel?):

- 1 zentrale Benutzeroberfläche (UI): Ein modernes, ästhetisches Dashboard mit Spielen (Covers, Titel, Plattform-Icon).

- Lokale Spiele-Erkennung (Core Feature): Automatisches Auslesen von lokal installierten Spielen (z. B. über Windows-Registrierung, Manifest-Dateien oder Standard-Installationspfade von Steam/Epic).

- Launch-Funktion: Klickbarer "Spielen"-Button, der das entsprechende Spiel direkt startet (z. B. via Custom Protocol URIs wie steam://run/<app_id>).

- Multi-Plattform-Anbindung: Integration von mindestens 2–3 großen Launchern als Prototyp (z. B. Steam & Epic Games).

- Such- & Filter-Funktion: Schnellsuche nach Spieletiteln und Filtern nach Plattform oder Installationsstatus.

Kompetenzen(warum?):

- **Desktop-App-Entwicklung:** Einarbeitung in ein Desktop-Framework wie **Electron** oder **Tauri** (um Web-Technologien wie HTML/CSS/JS als eigenständige Windows-App zu verpacken).

- **System- & Datei-Interaktion:** Lernen, wie eine App mit dem Windows-Betriebssystem kommuniziert (Dateipfade durchsuchen, Registry auslesen, System-Befehle zum Starten von Programmen ausführen).

- **Schnittstellen & API-Handling:** Einbinden von APIs oder lokalen Config-Dateien (z. B. Steam Web API, VDF-Parsing für Steam-Manifeste), um Bibliotheken und Game-Artworks dynamisch zu laden.